

1. Introduction to React.js

Question 1: What is React.js? How is it different from other JavaScript frameworks and libraries?

React.js is a **JavaScript library** developed by Facebook used to build **user interfaces**, especially **single-page applications (SPA)**. It focuses mainly on the **view layer** of an application.

React is different from other JavaScript frameworks because it:

- Uses a **component-based architecture**
 - Uses a **Virtual DOM** for better performance
 - Supports **one-way data binding**
 - Allows creation of **reusable UI components**
 - Is lightweight compared to full frameworks like Angular
-

Question 2: Core principles of React (Virtual DOM & Component-based Architecture)

Virtual DOM

React creates a virtual copy of the real DOM. When data changes, React compares the new virtual DOM with the old one and updates only the changed parts in the real DOM. This makes applications faster and more efficient.

Component-based Architecture

React applications are built using components. Each component is an independent, reusable block of UI that manages its own logic and structure.

Question 3: Advantages of using React.js

- Fast performance due to Virtual DOM
- Reusable components
- Easy to learn and use
- Strong community support
- Better UI management

- . Used by large companies like Facebook, Netflix, Instagram
-

2. JSX (JavaScript XML)

Question 1: What is JSX in React.js? Why is it used?

JSX is a syntax extension that allows writing **HTML-like code inside JavaScript**. It makes React code easier to read, write, and understand.

Question 2: JSX vs Regular JavaScript

JSX looks similar to HTML but is written inside JavaScript.

Yes, JavaScript expressions **can be written inside JSX**.

Question 3: Importance of curly braces {} in JSX

Curly braces are used to embed JavaScript expressions such as:

- Variables
- Functions
- Calculations inside JSX.

3. Components (Functional & Class Components)

Question 1: What are components?

Difference between functional and class components

Components are **reusable building blocks** of a React application.

Functional Components

- Simple JavaScript functions
- Use React Hooks
- No render() method

Class Components

- . ES6 classes
 - . Use render() method
 - . Can use lifecycle methods
-

Question 2: Passing data using props

Props are used to pass data from a **parent component to a child component**. Props are read-only and cannot be modified by the child component.

Question 3: Role of render() in class components

The render() method returns JSX and defines what should be displayed on the screen.

4. Props and State

Question 1: Props vs State

- . Props are passed from parent to child

- State is managed inside the component
 - Props are read-only, state is changeable
-

Question 2: What is state?

State is an object that stores **dynamic data** of a component. When state changes, the component re-renders automatically.

Question 3: Why `this.setState()` is used?

`this.setState()` is used in class components to update state safely and trigger re-rendering.

5. Handling Events in React

Question 1: Events in React & Synthetic Events

React uses **Synthetic Events**, which are wrappers around native browser events and provide consistent behavior across browsers.

Question 2: Common event handlers

- onClick – button click
- onChange – input change
- onSubmit – form submission

Question 3: Binding event handlers in class components

Binding is required to ensure the correct this context when methods are used as event handlers.

6. Conditional Rendering

Question 1: What is conditional rendering?

Conditional rendering means displaying UI elements based on certain conditions.

Question 2: Methods used

- . if-else statements
 - . Ternary operator (condition ? true : false)
 - . Logical AND (&&)
-

7. Lists and Keys

Question 1: Rendering lists & importance of keys

Lists are rendered using the `map()` function. Keys help React identify which items have changed, added, or removed.

Question 2: What are keys?

Keys are unique identifiers assigned to list items. Without keys, React may cause performance issues and incorrect UI updates.

8. Forms in React

Question 1: Handling forms & controlled components

In controlled components, form input values are controlled using React state.

Question 2: Controlled vs Uncontrolled components

- Controlled components use state
 - Uncontrolled components use direct DOM references
-

9. Lifecycle Methods (Class Components)

Question 1: Lifecycle methods & phases

Lifecycle phases:

1. Mounting
 2. Updating
 3. Unmounting
-

Question 2: Purpose of lifecycle methods

- `componentDidMount()`: runs after component loads
 - `componentDidUpdate()`: runs after updates
 - `componentWillUnmount()`: runs before removal
-

10. Hooks

Question 1: What are React hooks?

Hooks allow functional components to use state and lifecycle features.

Question 2: Problems solved by hooks

- Avoid class components
 - Reuse logic
 - Cleaner code
-

Question 3: `useReducer`

useReducer is used for managing complex state logic similar to Redux.

Question 4: useCallback & useMemo

These hooks are used to improve performance by avoiding unnecessary re-renders.

Question 5: Difference between useCallback & useMemo

- useCallback memoizes functions
 - useMemo memoizes values
-

Question 6: useRef

useRef is used to access DOM elements and store values without re-rendering.

11. Routing in React

Question 1: What is React Router?

React Router enables navigation in single-page applications without page reload.

Question 2: Components

- BrowserRouter – wraps app
 - Route – defines paths
 - Link – navigation
 - Switch/Routes – renders matching route
-

12. JSON-Server & Firebase

Question 1: RESTful web services

REST is an architecture for communication between client and server using HTTP methods.

Question 2: JSON-Server

JSON-Server is a fake REST API used for frontend testing.

Question 3: fetch() / axios()

They are used to make HTTP requests to APIs.

Question 4: Firebase

Firebase is a backend platform that provides database, authentication, and hosting.

Question 5: Error & loading states

Handling loading and errors improves user experience and avoids application crashes.

13. Context API

Question 1: What is Context API?

Context API is used to manage global state and avoid prop drilling.

Question 2: createContext & useContext

They are used to create and consume shared state across components.

14. State Management

Question 1: Redux

Redux is a state management library that uses:

- . Actions
 - . Reducers
 - . Store
-

Question 2: Recoil vs Redux

Recoil is simpler, requires less boilerplate, and integrates easily with React.

